

ใบงานที่ 5 การสร้างโมเดลการเรียนรู้แบบไม่มีผู้สอนด้วย K-Means (Unsupervised Learning)

วัตถุประสงค์การเรียนรู้

- อธิบายความจำเป็นและเขียนคำสั่งเพื่อปรับสเกลข้อมูล (Feature Scaling) ได้
- วิเคราะห์หาจำนวนกลุ่ม (K) ที่เหมาะสมด้วย Elbow Method และสร้างโมเดล K-Means เพื่อจัดกลุ่มข้อมูลได้
- ประยุกต์ใช้โมเดลเพื่อพยากรณ์ข้อมูลใหม่และกำหนดกลยุทธ์การตลาดได้อย่างเหมาะสม

เครื่องมือและอุปกรณ์

- เครื่องคอมพิวเตอร์ที่ติดตั้งโปรแกรม PyCharm IDE
- ไฟล์ข้อมูลดิบ lab3.csv
- เอกสารใบความรู้หน่วยที่ 5
- แบบฟอร์มบันทึกผลการปฏิบัติงาน (ท้ายใบงาน)

ลำดับขั้นตอนการปฏิบัติงาน

คำชี้แจง: ให้ผู้เรียนเขียนโค้ดลงในไฟล์ตามขั้นตอนต่อไปนี้

ขั้นตอนที่ 1: การเตรียมและปรับสเกลข้อมูล (Data Preparation)

```
# 1. นำไฟล์ lab3.csv จากโฟลเดอร์ Data management มาเป็น Dataset สำหรับ AI แบ่งกลุ่มลูกค้า
# 2. สร้างไฟล์ใหม่ชื่อ prepare_data.py
import pandas as pd
from sklearn.preprocessing import StandardScaler
import joblib

# 3. โหลดข้อมูลจากไฟล์ lab3.csv
print("กำลังโหลดข้อมูล...")
df = pd.read_csv('lab3.csv')
```

```

data = df[['Recency', 'Frequency', 'Monetary']].dropna()

# 4. สร้างตัวแปร Scaler และทำการปรับสเกล (Scaling)
print("กำลังปรับสเกลข้อมูล...")
scaler = StandardScaler()
data_scaled = scaler.fit_transform(data)

# 5. แปลงผลลัพธ์กลับเป็น DataFrame
df_scaled = pd.DataFrame(data_scaled, columns=['Recency', 'Frequency', 'Monetary'])

# 6. บันทึกไฟล์ข้อมูล (csv) และ โมเดลการสเกล (pkl)
df_scaled.to_csv('customer_scaled.csv', index=False)
joblib.dump(scaler, 'customer_scaler.pkl')

print("บันทึกไฟล์สำเร็จ: customer_scaled.csv และ customer_scaler.pkl")

```

ขั้นตอนที่ 2: การหาค่า K ที่เหมาะสม (Elbow Method)

```

# 1. สร้างไฟล์ใหม่ชื่อ find_k.py
import pandas as pd
from sklearn.cluster import KMeans

# 2. โหลดข้อมูลที่สเกลแล้ว
df = pd.read_csv('customer_scaled.csv')
X = df.values

# 3. วนลูปหาค่า Inertia ตั้งแต่ K=1 ถึง 10
print("--- ผลการทดสอบ Elbow Method ---")
for k in range(1, 11):
    # กำหนด n_init=10 เพื่อความแม่นยำ
    model = KMeans(n_clusters=k, random_state=42, n_init=10)
    model.fit(X)

```

```
# แสดงค่า Inertia ของแต่ละ K
print(f"K = {k}, Inertia = {model.inertia_:.2f}")
```

ขั้นตอนที่ 3: การสร้างโมเดลและระบุชื่อกลุ่ม (Model Training & Profiling)

```
# 1. สร้างไฟล์ใหม่ชื่อ train_model.py
import pandas as pd
from sklearn.cluster import KMeans
import joblib
import pickle

# 2. โหลดข้อมูล
df = pd.read_csv('customer_scaled.csv')
X = df.values

# 3. สร้างโมเดล K-Means (กำหนด K=3 ตามผลจาก Elbow Method)
optimal_k = 3
kmeans = KMeans(n_clusters=optimal_k, random_state=42, n_init=10)
df['Cluster'] = kmeans.fit_predict(X)

# 4. วาดรูปแสดงค่าเฉลี่ยของแต่ละกลุ่ม (Centroid Analysis)
print("\nn== วิเคราะห์พฤติกรรมกลุ่ม ==")
for i in range(optimal_k):
    cluster_data = df[df['Cluster'] == i]
    print(f"กลุ่ม {i}: R={cluster_data['Recency'].mean():.2f}, "
          f"F={cluster_data['Frequency'].mean():.2f}, "
          f"M={cluster_data['Monetary'].mean():.2f}")

# 5. ระบุชื่อกลุ่ม (Naming) ตามเงื่อนไข
cluster_names = {}
for i in range(optimal_k):
    cluster_data = df[df['Cluster'] == i]
    avg_r = cluster_data['Recency'].mean()
```

```

avg_f = cluster_data['Frequency'].mean()
avg_m = cluster_data['Monetary'].mean()

# Logic การตั้งชื่อ: VIP, Churned, หรือ General
if avg_m > 0.8 and avg_f > 0.7:
    cluster_names[i] = "💎 VIP (ซื้อเยอะ-บ่อย)"
elif avg_r > 1.0:
    cluster_names[i] = "⚠️ Churned (หายไปนาน)"
else:
    cluster_names[i] = "😊 General (ทั่วไป)"

# 6. บันทึกโมเดลและชื่อกลุ่ม
joblib.dump(kmeans, 'customer_kmeans.pkl')
with open("cluster_names.pkl", "wb") as f:
    pickle.dump(cluster_names, f)

print("บันทึกโมเดลเรียบร้อยแล้ว")

```

ขั้นตอนที่ 4: การพยากรณ์และการกำหนดกลยุทธ์ (Prediction & Strategy)

```

# 1. สร้างไฟล์ใหม่ชื่อ run_model.py
import pandas as pd
import joblib
import pickle

# 2. โหลดโมเดล, Scaler และชื่อกลุ่ม
kmeans = joblib.load('customer_kmeans.pkl')
scaler = joblib.load('customer_scaler.pkl')
with open("cluster_names.pkl", "rb") as f:
    names = pickle.load(f)

# 3. สร้างฟังก์ชันพยากรณ์ (Predict Function)
def predict_user(recency, frequency, monetary):

```

```

# เตรียมข้อมูลและ *Scale ด้วย Scaler ตัวเดิม*
user_data = pd.DataFrame([[recency, frequency, monetary]],
                           columns=['Recency', 'Frequency', 'Monetary'])
user_scaled = scaler.transform(user_data)

# พยากรณ์กลุ่ม
cid = kmeans.predict(user_scaled)[0]
return names.get(cid, "Unknown")

# 4. กำหนดกลยุทธ์ (Strategy Logic)
def get_strategy(group_name):
    if "VIP" in group_name:
        return "เสนอสินค้า Premium + ส่งฟรี"
    elif "Churned" in group_name:
        return "ส่งคูปองส่วนลด 50% เพื่อดึงกลับมา"
    else:
        return "เสนอโปรโมชั่น ชื้อ 2 แถม 1"

# 5. ทดสอบพยากรณ์
customer_a = [1, 25, 15000] # ลูกค้า A
group_a = predict_user(*customer_a)
print(f"ลูกค้า A: อยู่กลุ่ม {group_a} -> กลยุทธ์: {get_strategy(group_a)}")

customer_b = [365, 1, 100] # ลูกค้า B
group_b = predict_user(*customer_b)
print(f"ลูกค้า B: อยู่กลุ่ม {group_b} -> กลยุทธ์: {get_strategy(group_b)}")

```

แบบบันทึกผลการปฏิบัติงาน

ส่วนที่ 1: ผลการเตรียมและปรับสเกลข้อมูล

ตรงกับวัตถุประสงค์การเรียนรู้ข้อที่ 1: (อธิบายความจำเป็นและเขียนคำสั่งปรับสเกลข้อมูล)

1. จำนวนข้อมูล (Rows):
 - ข้อมูลที่โหลดเข้ามาทั้งหมดมีจำนวน: 300 แถว
2. การเปรียบเทียบค่าข้อมูล (Data Scaling Verification): จงเลือกดูข้อมูลลูกค้าคนแรก (Row 0) และบันทึกค่าที่เปลี่ยนแปลงไป:
 - ก่อนปรับสเกล (Raw Data): Recency = 5, Monetary = 15000
 - หลังปรับสเกล (Scaled Data): Recency = -0.8196, Monetary = 2.5253

ส่วนที่ 2: ผลการหาจำนวนกลุ่ม

ตรงกับวัตถุประสงค์การเรียนรู้ข้อที่ 2: (วิเคราะห์หาจำนวนกลุ่ม (K) ที่เหมาะสมด้วย Elbow Method)

1. จำนวนข้อมูล (Rows):
 - K = 1, Inertia = 900.00
 - K = 2, Inertia = 399.03
 - K = 3, Inertia = 170.08
 - K = 4, Inertia = 129.59
 - K = 5, Inertia = 98.82
2. การตัดสินใจ:
 - จากข้อมูลควรเลือกแบ่งลูกค้าออกเป็น 3 กลุ่ม

ส่วนที่ 3: ผลการสร้างโมเดลและตั้งชื่อกลุ่ม

ตรงกับวัตถุประสงค์การเรียนรู้ข้อที่ 3: (ประยุกต์ใช้โมเดลเพื่อกำหนดกลยุทธ์การตลาด)

1. ค่าเฉลี่ยของแต่ละกลุ่ม (Centroid Analysis):

กลุ่มที่ (Cluster ID)	Recency (Mean)	Frequency (Mean)	Monetary (Mean)
กลุ่ม 0	-0.46	-0.15	-0.50
กลุ่ม 1	1.50	-1.01	-0.68
กลุ่ม 2	-0.73	1.26	1.50

2. เงื่อนไขการตั้งชื่อกลุ่ม (Naming Logic): จงระบุเงื่อนไข IF-ELSE ใน Code ที่ใช้กำหนดชื่อกลุ่ม

- กลุ่ม VIP: ต้องมีค่า $avg_m > 0.8$ และ $avg_f > 0.7$
- กลุ่ม Churned: ต้องมีค่า $avg_r > 1.0$
- กลุ่ม General: คือลูกค้ากลุ่มที่เหลือ

ส่วนที่ 4: ผลการพยากรณ์และกลยุทธ์

ตรงกับวัตถุประสงค์การเรียนรู้ข้อที่ 3: (ประยุกต์ใช้โมเดลเพื่อพยากรณ์ข้อมูลใหม่และกำหนดกลยุทธ์)

1. ทดสอบลูกค้า A (1, 25, 15000):

- กลุ่มที่พยากรณ์ได้:  VIP (ซื้อเยอะ -บ่อย)
- กลยุทธ์ที่ระบบเลือก: เสนอสินค้า Premium + ส่งฟรี

2. ทดสอบลูกค้า B (365, 1, 100):

- กลุ่มที่พยากรณ์ได้:  Churned (หายไปนาน)
- กลยุทธ์ที่ระบบเลือก: ส่งคูปองส่วนลด 50% เพื่อดึงกลับมา

ส่วนที่ 5: คำถามท้ายการทดลอง

ตรงกับวัตถุประสงค์การเรียนรู้ข้อที่ 1: (อธิบายความจำเป็นของการสเกลข้อมูล)

1. ในไฟล์ `prepare_data.py` ทำไมเราต้องบันทึกไฟล์ `customer_scaler.pkl` แยกออกมา? หากลืมนบันทึกไฟล์นี้ จะส่งผลอย่างไรเมื่อนำไปใช้จริงในไฟล์ `run_model.py`?

แนวคำตอบ: เราต้องบันทึก Scaler เอาไว้ เพื่อให้มั่นใจว่าเมื่อมี "ข้อมูลลูกค้าใหม่" เข้ามาในระบบ ข้อมูลนั้นจะถูกนำไปปรับสเกล (Transform) ด้วย มาตรฐานเดียวกัน (ค่า Mean และ SD) กับที่เคยใช้สอนโมเดลไว้ตั้งแต่ตอนแรก หากลืมนบันทึกตัวแปรนี้ โมเดลจะไม่สามารถแปลความหมายของตัวเลขดิบๆ (เช่น Monetary 15,000) ได้อย่างถูกต้อง หรือถ้าสร้าง Scaler ขึ้นมาใหม่โดยใช้เพียงข้อมูลของลูกค้าคนเดียว ค่าเฉลี่ยและส่วนเบี่ยงเบนมาตรฐานก็จะผิดเพี้ยน ทำให้ผลลัพธ์การพยากรณ์กลุ่มลูกค้าผิดพลาดไปโดยสิ้นเชิง